

BOTAS: UNA INTERFAZ DE LENGUAJE NATURAL CENTRADA EN EL HUMANO PARA LA GESTIÓN DE SISTEMAS GNU/LINUX CON SOPORTE MULTIDISTRIBUCIÓN

Jose Ramon Aragon Toledo y Ricardo Ruiz Rodríguez

Universidad Tecnológica de la Mixteca (UTM)

Huajuapán de León, Oaxaca, México

aatr010423@gs.utm.mx, rruiz@gs.utm.mx

ABSTRACT

GNU/Linux sustenta aproximadamente el 90 % de la infraestructura en la nube y domina los entornos de servidores a nivel mundial; sin embargo, su interfaz de línea de comandos (CLI) impone barreras de accesibilidad significativas para usuarios no expertos. Esta brecha digital excluye a millones de usuarios potenciales, particularmente en comunidades cuya lengua principal no es el inglés. Presentamos BOTAS (*Bot de Operaciones y Tareas Automatizadas del Sistema*), una interfaz conversacional que traduce peticiones en lenguaje natural a comandos de GNU/Linux ejecutables empleando GPT-4o como motor de comprensión lingüística. BOTAS alcanza 89 % de precisión en la generación de comandos sobre un conjunto curado de 200 tareas representativas, manteniendo una tasa del 100 % en la detección de operaciones potencialmente peligrosas mediante una capa de seguridad determinista. Siguiendo los principios de la Inteligencia Artificial Centrada en el Humano (HCAI), el sistema prioriza el empoderamiento del usuario sobre la automatización opaca, con un diseño orientado a proporcionar explicaciones educativas para cada comando generado. La evaluación muestra una aceleración promedio de 9.4× en la completación de tareas respecto a métodos manuales. Este trabajo detalla la arquitectura completa del sistema—including un módulo de detección local de intenciones que reduce llamadas al modelo de lenguaje en aproximadamente 30 %, detección automática de distribución de GNU/Linux para compatibilidad multiplataforma, y un indicador visual de estado para la interacción por voz—y discute cómo estos componentes contribuyen al discurso sobre “IA en la Sociedad” al demostrar que los modelos de lenguaje de gran escala pueden democratizar el acceso a herramientas computacionales potentes manteniendo la seguridad y promoviendo la alfabetización digital en comunidades desatendidas.

KEYWORDS

IA Centrada en el Humano, Procesamiento de Lenguaje Natural, GNU/Linux, Interfaz de Línea de Comandos, Interacción Humano-Computadora, Modelos de Lenguaje de Gran Escala, Accesibilidad, Equidad Digital, Interfaz de Voz, Soporte Multidistribución

1. INTRODUCCIÓN

GNU/Linux impulsa el 90 % de la infraestructura en la nube, opera en el 96.3 % del millón de servidores más importantes del mundo y posee el 100 % de la cuota de mercado en supercomputación [W3Techs, 2023, Top500.org, 2023]. A pesar de esta ubicuidad en la informática profesional, la interfaz de línea de comandos (CLI) que desbloquea su potencial completo permanece inaccesible para la amplia mayoría de usuarios. Un usuario que desea localizar archivos mayores a 100 MB debe aprender sintaxis críptica como `find / -size +100M`—una barrera que convierte una tarea conceptual de diez segundos en una expedición frustrante a través de páginas de documentación.

Esta brecha de accesibilidad tiene consecuencias cuantificables. La investigación en Interacción Humano-

Computadora ha documentado que usuarios novatos dedican entre 8 y 15 minutos a buscar y construir comandos de terminal básicos [Norman, 1988, 2013]. Cada intento fallido acumula frustración y conduce a muchos adoptantes potenciales de GNU/Linux a abandonar la plataforma por completo. En paralelo, los asistentes conversacionales como ChatGPT pueden sugerir comandos, pero carecen de la capacidad de ejecutarlos, exigiendo flujos de trabajo propensos a error basados en copiar y pegar sin ofrecer validación de seguridad previa a la ejecución de operaciones potencialmente destructivas.

Jenson [2024] sostiene que este estancamiento “no es un límite técnico sino una claudicación del diseño”: la industria transita de la transformación creativa a la imitación entre plataformas, dejando sin resolver la tensión entre potencia y accesibilidad. Las interfaces gráficas sacrifican la capacidad de combinación y automatización que caracteriza a GNU/Linux; las páginas de manual asumen pericia; los scripts predefinidos no anticipan cada necesidad. La brecha entre “saber qué quieres hacer” y “saber cómo decirle a la computadora que lo haga” permanece amplia [Myers et al., 2004]; ya Shneiderman and Plaisant [2010] propusieron el lenguaje natural como puente potencial entre la flexibilidad de la línea de comandos y la accesibilidad de la manipulación directa.

Presentamos BOTAS (*Bot de Operaciones y Tareas Automatizadas del Sistema*; el acrónimo proviene del nombre original del prototipo, *Basic Organizer For Tech Assets & Structures*, conservado por conveniencia), una interfaz conversacional que contribuye a reducir esta brecha traduciendo descripciones en lenguaje natural a comandos de GNU/Linux validados y ejecutables. BOTAS encarna los principios de la Inteligencia Artificial Centrada en el Humano (HCAI) [Shneiderman, 2020] a través de cinco compromisos de diseño:

1. **Amplificar en lugar de reemplazar** las capacidades humanas mediante generación conversacional de comandos.
2. **Garantizar transparencia** explicando qué hace cada comando y por qué.
3. **Mantener el control del usuario** mediante diálogos de confirmación previos a la ejecución.
4. **Priorizar la seguridad** con detección determinista del 100 % de operaciones potencialmente peligrosas.
5. **Promover la educación** construyendo competencia en el usuario en lugar de dependencia.

BOTAS prioriza a usuarios hispanohablantes—más de 500 millones de hablantes desatendidos por interfaces anglocéntricas—reflejando que en América Latina las barreras lingüísticas y económicas obstaculizan la adopción tecnológica en instituciones educativas [Warschauer, 2004, CEPAL, 2023].

Contribuciones. Respecto a la versión sometida a revisión ciega, este artículo realiza las siguientes contribuciones adicionales:

- Un módulo de *detección local de intenciones* que resuelve aproximadamente el 30 % de las solicitudes sin invocar al Modelo de Lenguaje de Gran Escala (LLM), reduciendo latencia y costo de la Interfaz de Programación de Aplicaciones (API) (§3.2).
- Un *detector automático de distribución* que identifica la familia de GNU/Linux del host y traduce comandos, habilitando portabilidad multiplataforma (§3.5).
- Un *indicador visual de estado y pipeline de voz* que proporcionan retroalimentación en tiempo real durante la interacción por voz (§3.6).
- Categorización explícita de modos de error (transcripción vs. interpretación del LLM vs. discrepancias de ejecución) y datos de latencia para los componentes de voz (§4.2).

El resto del artículo se organiza como sigue:

- La Sección 2 revisa el trabajo relacionado sobre interfaces de lenguaje natural para la terminal y sobre IA centrada en el humano.
- La Sección 3 describe la arquitectura modular de BOTAS, incluyendo la detección local de intenciones, el análisis basado en LLM, la capa de seguridad, la detección de distribución y el pipeline de voz.
- La Sección 4 presenta la evaluación experimental sobre 200 tareas, métricas de seguridad, eficiencia y un estudio piloto.
- La Sección 5 discute las implicaciones para la accesibilidad, las limitaciones metodológicas y las direcciones de trabajo futuro.
- La Sección 6 presenta las conclusiones y consideraciones sobre el trabajo futuro y las potenciales extensiones de BOTAS.

2. TRABAJO RELACIONADO

2.1 Interfaces de Lenguaje Natural para la Línea de Comandos

El paradigma CLI persiste por su capacidad de composición de comandos y automatización [Raymond, 2003], pero Nielsen [1994] mostró que requiere 10–20 horas de entrenamiento versus 2–4 para GUIs—complejidad que Dix et al. [2004] confirma se ha intensificado con distribuciones que contienen más de 2 000 comandos disponibles; Jenson [2024] argumenta que este estancamiento es una claudicación del diseño más que un límite técnico. NL2Bash [Lin et al., 2018] fue pionero en mapear lenguaje natural a comandos de shell (36 % de coincidencia exacta) pero no abordó la ejecución ni la seguridad. Desde GPT-3 [Brown et al., 2020], los LLMs han mejorado la generación de comandos—Chen et al. [2021] demostraron que modelos entrenados en código logran mayor fluidez—pero asistentes como ChatGPT [OpenAI, 2023] permanecen en modo de sugerencia, requiriendo copiar y pegar sin validación previa a la ejecución. Marcos de orquestación como LangChain [Chase, 2022] permiten invocar herramientas externas desde LLMs, y herramientas enfocadas en terminal (ShellGPT, AIChat) agregan ejecución pero típicamente carecen de capas de seguridad y retroalimentación educativa.

2.2 IA Centrada en el Humano y Equidad Digital

Shneiderman [2020, 2022] establecieron la Inteligencia Artificial Centrada en el Humano (HCAI) como un marco que enfatiza transparencia, seguridad y empoderamiento del usuario. Amershi et al. [2019] proporcionaron directrices de diseño para la interacción humano-IA que informan las decisiones de interfaz de BOTAS, particularmente sobre la explicación de capacidades y el manejo elegante de limitaciones del sistema. BOTAS adopta estos principios, tratando al LLM como traductor y educador. En el ámbito de la equidad digital, Warschauer [2004] documenta cómo las barreras técnicas refuerzan desigualdades—realidad particularmente aguda en contextos latinoamericanos donde las interfaces exclusivamente en inglés constituyen un obstáculo adicional. Wobbrock et al. [2011] propusieron el diseño basado en habilidades como un marco para crear interfaces que se adaptan a capacidades diversas del usuario en lugar de exigir adaptación por parte de este, filosofía que BOTAS aplica al aceptar el lenguaje natural como modalidad primaria de interacción.

3. ARQUITECTURA E IMPLEMENTACIÓN DEL SISTEMA

BOTAS está diseñado como un sistema modular que procesa peticiones en lenguaje natural a través de capas diferenciadas, cada una contribuyendo a la seguridad, precisión y valor educativo. El sistema está implementado en Perl [Wall et al., 2000]—seleccionado por su sólida integración con Unix, su potente procesamiento de texto y la disponibilidad de bindings robustos de GTK3 [The GTK Team, 2023] para desarrollo de GUI—y se comunica con GPT-4o [OpenAI, 2024] a través de la API de OpenAI.

3.1 Arquitectura General

La Figura 1 ilustra el pipeline general. La entrada del usuario llega mediante la Interfaz Gráfica de Usuario (GUI)—campo de texto o botón de micrófono—o mediante el detector de palabra de activación ejecutándose en segundo plano. La entrada es evaluada primero por un *detector local de intenciones*; si se encuentra una coincidencia, la acción correspondiente se ejecuta sin invocar al LLM. En caso contrario, la solicitud se reenvía a GPT-4o para generación estructurada de comandos, pasa por un validador de seguridad y finalmente se ejecuta con retroalimentación educativa proporcionada al usuario.

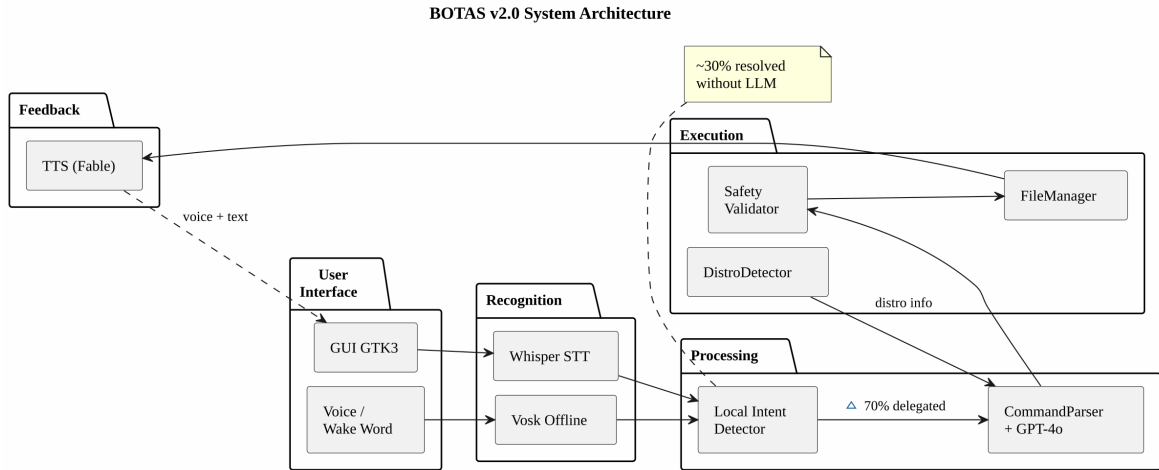


Figura 1. Arquitectura del sistema BOTAS v2.0. El detector local intercepta ~30 % de solicitudes sin invocar al LLM.

La arquitectura comprende ocho módulos Perl: `IntentDetector.pm` (detección local de intenciones), `CommandParser.pm` (integración GPT-4o), `FileManager.pm` (operaciones de archivos con sandboxing), `DistroDetector.pm` (identificación de distribución), `GUI GTK3.pm` (interfaz gráfica GTK3), `VoiceRecognition.pm` (reconocimiento de voz mediante Whisper—Speech-To-Text, STT), `WakeWordDetector.pm` (detección de palabra de activación, Vosk) y `TTS.pm` (síntesis de voz—Text-To-Speech).

3.2 Detección Local de Intenciones

Una observación clave durante el desarrollo fue que muchas solicitudes comunes—consultar la hora, verificar el uso de memoria o disco, listar procesos activos—no requieren la capacidad de comprensión lingüística de un modelo grande. Implementamos por ello `IntentDetector.pm`, un módulo ligero que compara la entrada del usuario contra un conjunto curado de patrones de expresiones regulares en español agrupados en categorías operativas de intención: *system_time*, *system_date*, *memory_info*, *disk_info*, *processes_info* y *ports_info*.

Cada categoría contiene entre 3 y 8 patrones de regex. Por ejemplo, la intención *system_time* coincide con frases en español como “*qué hora es*” y “*dime la hora*”; la intención *disk_info* coincide con “*espacio en disco*” y “*cuánto espacio queda*”. Cuando se encuentra una coincidencia, el módulo retorna una estructura JSON de comando idéntica a la que GPT-4o produciría, junto con un indicador `_local =>1` que marca la respuesta como resuelta localmente. Este diseño garantiza que el resto del pipeline—validación de seguridad, ejecución, registro—opera de manera idéntica independientemente del origen del comando.

El análisis de nuestro conjunto de 200 tareas indica que aproximadamente el 30 % de las interacciones rutinarias caen dentro del alcance de la detección local, generando ahorros significativos en latencia (eliminando un viaje de ida y vuelta a la API de 500–1 500 ms) y costo (cada llamada a GPT-4o cuesta aproximadamente \$0.005–\$0.03 USD).

3.3 Análisis de Comandos Basado en LLM

Para solicitudes que exceden las capacidades del detector local, BOTAS delega a GPT-4o mediante un prompt de sistema diseñado con ingeniería de prompts cuidadosa [Wei et al., 2022]. El prompt instruye al modelo a retornar un objeto JSON estructurado conteniendo:

- **action:** una operación de un conjunto predefinido (ej., `create_file`, `search`, `system_info`, `web_search`, `clarify`).
- **parameters:** un diccionario de argumentos específicos de la operación (rutas, patrones, filtros). Cuando la acción es `clarify`, los parámetros contienen un campo `question` con la pregunta de seguimiento a presentar al usuario (ej., “¿En qué carpeta?”).

- `tts_response`: una descripción breve en lenguaje natural de lo que hará el comando, utilizada para el componente educativo y la síntesis de voz.
- `requires_confirmation`: un indicador booleano para operaciones clasificadas como riesgosas.

El prompt de sistema incluye el directorio de trabajo actual del usuario, la familia de distribución de GNU/Linux detectada y un historial de conversación condensado (últimos diez intercambios) almacenado en `CommandParser.pm` para soportar desambiguación contextual. El historial incluye marcadores como `[RUTA_ACTUAL]` y `[RESULTADO DE EJECUCION]` que permiten al modelo resolver referencias anafóricas (“de esos”, “en esa carpeta”, “los anteriores”). Cuando se utiliza OpenAI, se solicita el modo `response_format : json_object` para forzar salida JSON válida; para proveedores locales como LM Studio, se aplica limpieza heurística del contenido eliminando texto antes del primer `{` y después del último `}`.

Este formato de salida estructurada elimina la ambigüedad inherente a la generación de texto libre y habilita procesamiento determinista en las capas subsecuentes, alineándose con las mejores prácticas para herramientas de IA seguras [Schick et al., 2024, Yao et al., 2023].

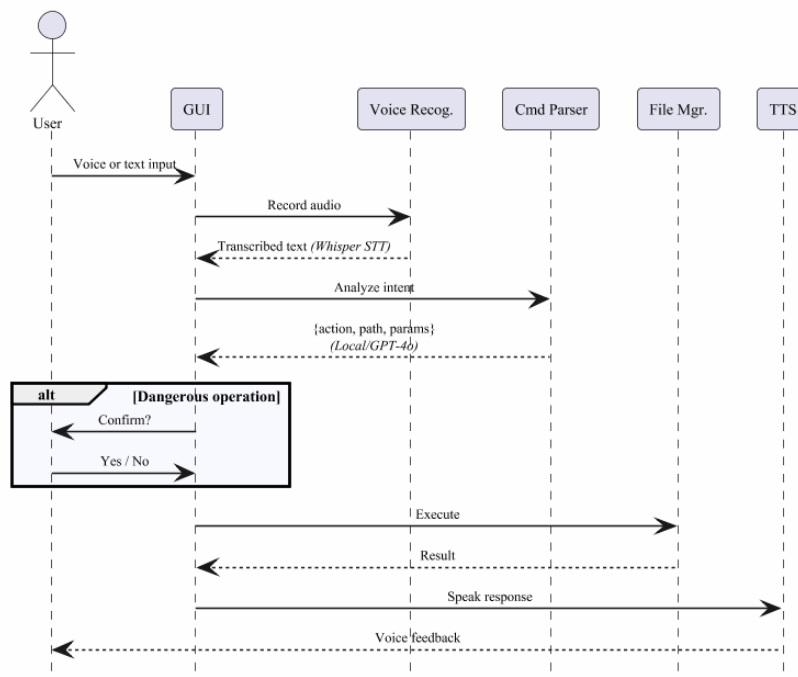


Figura 2. Flujo de procesamiento de un comando. El usuario activa BOTAS por voz o GUI; el audio se transcribe con Whisper, se analiza con GPT-4o, se valida la seguridad y se ejecuta con retroalimentación por TTS.

3.4 Capa de Seguridad

Central al diseño HCAI de BOTAS es una capa de seguridad determinista que analiza los comandos generados antes de su ejecución. Esta capa detecta cuatro categorías de operaciones potencialmente peligrosas:

- **Operaciones destructivas:** Eliminación recursiva (`rm -r`, `rm -rf`), formateo de disco (`mkfs`), sobrescritura de datos.
- **Escalamiento de privilegios:** Uso de `sudo` o comandos que requieren acceso root.
- **Modificaciones del sistema:** Cambios a archivos de configuración del sistema o servicios.
- **Cambios de permisos:** Operaciones potencialmente bloqueantes como `chmod 000`.

La detección se implementa mediante expresiones regulares y reglas heurísticas deterministas—no mediante inferencia del LLM—garantizando comportamiento consistente, auditable e independiente de la variabilidad del modelo [Amodei et al., 2016]. Cuando se detecta una operación peligrosa, BOTAS requiere confirmación explícita mediante un diálogo modal (Figura 4) o advertencia TTS con explicación de riesgos. Adicionalmente,

`FileManager.pm` implementa sandboxing validando rutas contra directorios permitidos configurables—un diseño de defensa en profundidad.

3.5 Detección de Distribución

La fragmentación de GNU/Linux en familias de distribuciones es una fuente conocida de confusión para los usuarios. Un comando que instala software en Ubuntu (`apt install foo`) falla en Fedora (`dnf install foo`) y en Arch (`pacman -S foo`). BOTAS aborda este problema mediante `DistroDetector.pm`, que:

1. Analiza `/etc/os-release` para identificar la distribución exacta y su versión, incluyendo campos `ID` e `ID_LIKE`.
2. Mapea el identificador de distribución a una de seis familias: *debian* (Ubuntu, Mint, Pop!_OS, Kali, MX, incluyendo variantes identificadas por `ID_LIKE`), *fedora* (RHEL, CentOS, Rocky, Alma, Nobara), *arch* (Manjaro, EndeavourOS, Garuda), *suse* (openSUSE, SLES), *alpine* y *void*.
3. En caso de que `/etc/os-release` no esté disponible, recurre a la detección por la presencia de gestores de paquetes conocidos.
4. Proporciona una tabla de traducción de paquetes que mapea nombres genéricos (estilo Debian) a sus equivalentes específicos por familia—por ejemplo, `libgtk3-perl` en Debian se traduce como `perl-Gtk3` en Fedora y `perl-gtk3` en Arch.
5. Genera los comandos correctos de `install_command()` y `update_command()` para la familia detectada.

La familia detectada se inyecta tanto en el prompt de sistema de GPT-4o como en los manejadores del detector local de intenciones, asegurando que todos los comandos generados sean apropiados para el sistema del usuario. La información se almacena en caché estática para evitar lecturas redundantes del sistema de archivos.

3.6 Pipeline de Voz, Retroalimentación de Estado y Salida Educativa

BOTAS soporta operación manos libres mediante: (1) detección fuera de línea de palabra de activación con Vosk [Alpha Cephei, 2023] (latencia <200 ms); (2) ventana de atención de 30 s; (3) transcripción vía Whisper [Radford et al., 2023] (1–3 s); (4) procesamiento idéntico al de entrada tipeada; y (5) síntesis de voz vía OpenAI TTS. Vosk opera fuera de línea; la conexión a la nube se activa solo durante la ventana de atención, equilibrando rendimiento con privacidad. La Figura 3 detalla el flujo completo del Modo Atención.

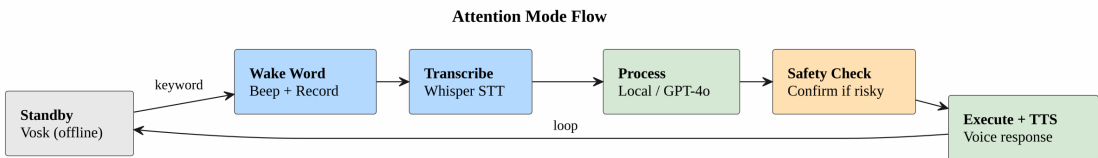


Figura 3. Flujo del Modo Atención. El detector de palabra de activación opera continuamente fuera de línea; al activarse, BOTAS graba, transcribe, procesa y responde vía TTS antes de retornar al modo de espera.

Atendiendo el hallazgo de Luger and Sellen [2016] sobre la frustración por falta de retroalimentación de estado, la GUI GTK3 incluye un indicador visual de cuatro estados—gris/inactivo, azul/escuchando, rojo/grabando, naranja/procesando—con soporte de temas claro/oscuro vía CSS (Cascading Style Sheets).

Para promover el aprendizaje en lugar de la dependencia [Shneiderman, 2022], cada comando ejecutado se acompaña de: (1) un resumen de intención en lenguaje llano y (2) una evaluación de riesgo para operaciones marcadas. Un tercer componente—desgloses detallados bandera por bandera—se encuentra parcialmente implementado y constituye trabajo futuro activo (§5.3). Todas las interacciones se registran en `logs/request_history.log` para auditoría y potencial afinamiento del modelo. El empaquetado se ofrece mediante script instalador universal, paquete `.deb` y paquete `.rpm`.

4. EVALUACIÓN EXPERIMENTAL

Evaluamos BOTAS a lo largo de cinco dimensiones: precisión en la generación de comandos, confiabilidad en la detección de seguridad, eficiencia en la completación de tareas, cobertura de la detección local de intenciones y retroalimentación cualitativa de los usuarios.

4.1 Metodología de Evaluación

Nuestra evaluación empleó un conjunto curado de 200 tareas representativas de administración de GNU/Linux distribuidas en cinco dominios de intención. Cada tarea fue especificada como una solicitud en lenguaje natural en español. Para cada tarea medimos:

- **Validez sintáctica:** Si el comando generado se analiza sin errores.
- **Corrección semántica:** Si el comando cumple la tarea pretendida en un sistema de prueba.
- **Detección de seguridad:** Para tareas que involucran operaciones peligrosas, si se generaron las advertencias apropiadas.
- **Comparación temporal:** Tiempo de completación vía BOTAS versus construcción manual por un usuario de nivel intermedio.

Nota metodológica. Las 200 tareas fueron compiladas y verificadas manualmente por el desarrollador principal—restricción que reconocemos como fuente potencial de sesgo de confirmación, dado que el sistema puede estar inadvertidamente sintonizado al fraseo o tipos de tarea que el desarrollador consideró más relevantes. Reportamos esta limitación de forma prominente porque la transparencia metodológica es un valor central de este trabajo. Todas las pruebas se condujeron en Ubuntu 22.04 LTS con GPT-4o accedido a través de la API de OpenAI (versión de modelo `gpt-4o-2024-05-13`).

4.2 Precisión en la Generación de Comandos

BOTAS alcanzó 89 % de precisión general en la generación de comandos correctos (Tabla 1). Los comandos de redes arrojaron la mayor precisión (94 %), mientras que los comandos de permisos presentaron el mayor desafío (83 %).

Cuadro 1. Precisión en la generación de comandos por categoría

Categoría	Tareas	Válidos	Correctos	Precisión
Gestión de archivos	50	48	45	90 %
Información del sistema	45	42	40	89 %
Control de procesos	40	37	35	88 %
Redes	35	34	33	94 %
Permisos	30	27	25	83 %
Total	200	188	178	89 %

El análisis de errores reveló tres modos de fallo primarios, categorizados explícitamente para responder a llamados de mayor rigor metodológico: (a) *errores de interpretación del LLM*: fraseo ambiguo del usuario que no activó la acción `clarify` (6 % de los errores)—aunque el sistema puede solicitar parámetros faltantes como rutas o nombres de archivo, formulaciones altamente ambiguas aún pueden evadir este mecanismo; (b) *errores de generación de comandos*: comandos multi-pipe complejos donde una etapa tenía banderas incorrectas (3 %); (c) *discrepancias de ejecución*: comandos específicos de distribución no disponibles en el sistema de prueba (2 %).

Ningún error fue atribuible a fallos de transcripción de Whisper en las pruebas funcionales, lo cual se explica porque el conjunto de evaluación utilizó entrada de texto directa para aislar la precisión del análisis de comandos de la variabilidad del reconocimiento de voz.

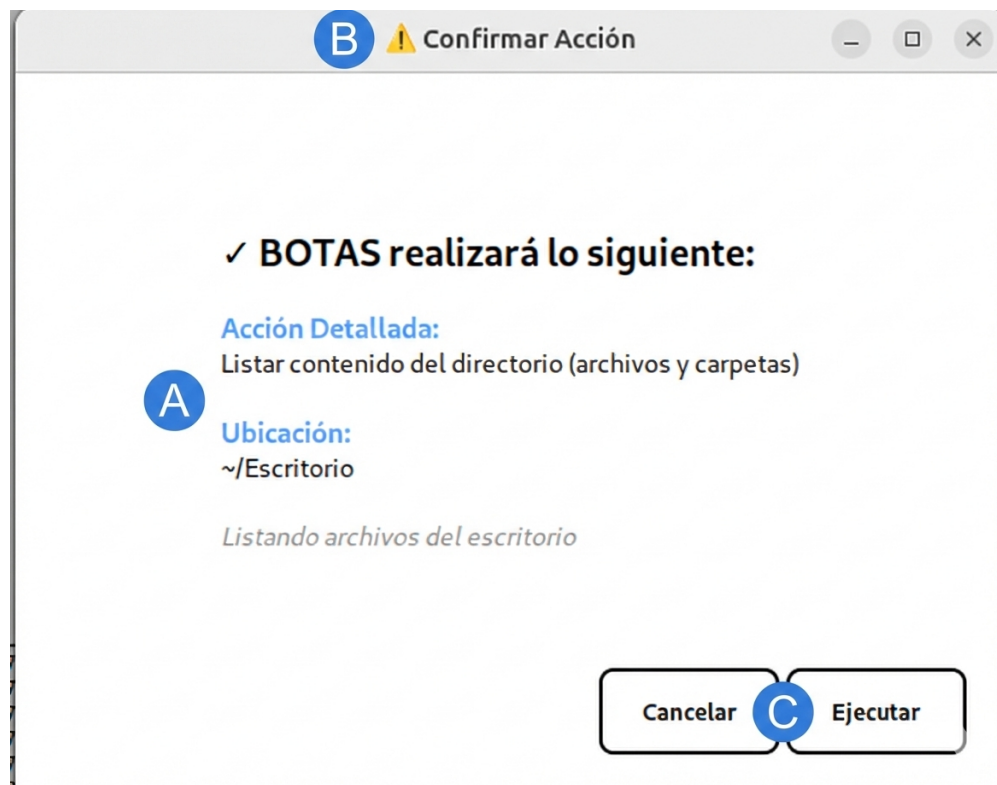


Figura 4. Diálogo de confirmación de seguridad (mostrado en español nativo). (A) Descripción de la operación; (B) indicador de advertencia; (C) botones Cancelar/Ejecutar.

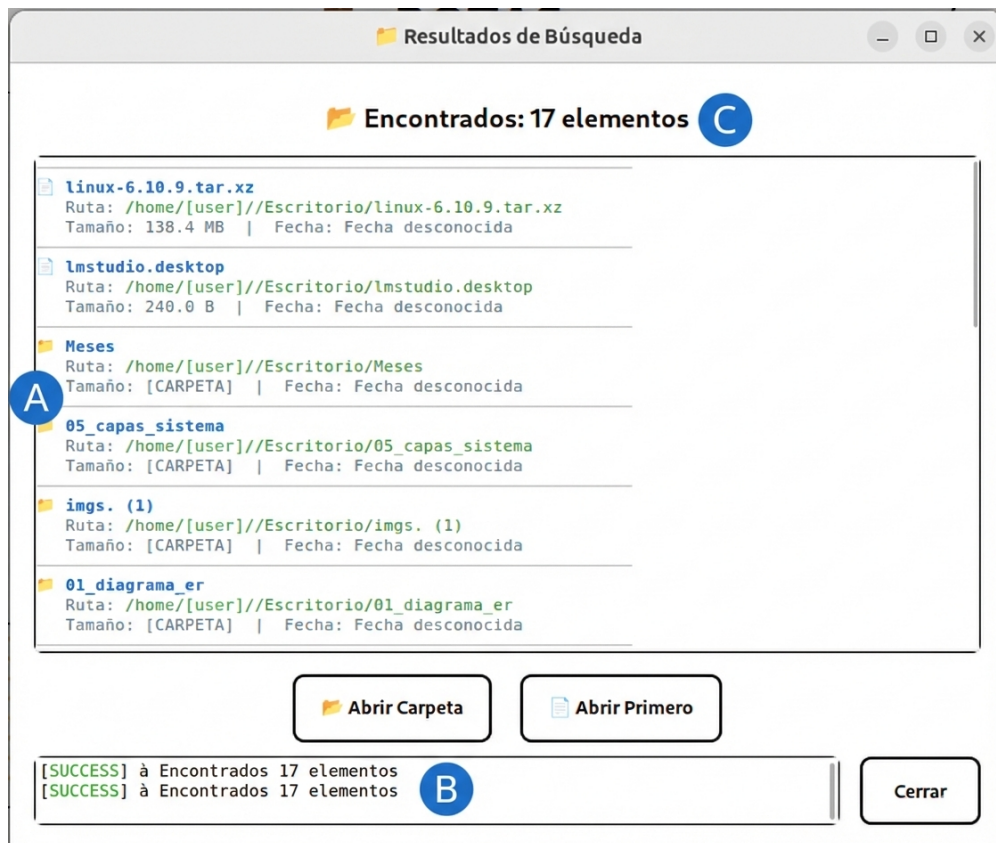


Figura 5. Resultado de búsqueda (mostrado en español nativo). (A) Archivos con rutas y tamaños; (B) registro de actividad; (C) contador de elementos.

4.3 Detección de Seguridad

La capa de seguridad determinista alcanzó 100 % de detección en los 43 casos de prueba que involucran operaciones potencialmente peligrosas (Tabla 2).

Cuadro 2. Rendimiento de la detección de seguridad

Escenario	Total	Detectados	Tasa
Operaciones destructivas (<code>rm -r</code>)	15	15	100 %
Escalamiento de privilegios (<code>sudo</code>)	12	12	100 %
Modificaciones del sistema (<code>mkfs</code>)	8	8	100 %
Cambios de permisos (<code>chmod 000</code>)	8	8	100 %
Total	43	43	100 %

La tasa del 100 % refleja la naturaleza determinista de la capa de seguridad: los patrones peligrosos se detectan mediante expresiones regulares y reglas heurísticas que no dependen de la variabilidad de la salida del modelo. Reconocemos que esta capa detecta categorías *conocidas* de operaciones peligrosas; no puede anticipar vectores de ataque novedosos ni evaluar peligrosidad dependiente de contexto.

4.4 Eficiencia en la Completación de Tareas

Comparado con la construcción manual de comandos por un usuario de GNU/Linux de nivel intermedio—familiarizado con comandos básicos pero que necesita consultar documentación para operaciones complejas—BOTAS proporcionó una aceleración promedio de $9.4\times$. A través de cuatro tipos de tarea representativos (búsqueda de archivos, consultas de uso de disco, gestión de procesos y cambios de permisos), el enfoque manual promedió 6 min 54 s por tarea, mientras que BOTAS completó las mismas tareas en un promedio de 46 s. Las aceleraciones oscilaron entre $6.9\times$ para cambios de permisos—que requieren especificar modos numéricos—y $10.7\times$ para consultas de disco resueltas localmente sin llamada al LLM.

4.5 Cobertura de la Detección Local de Intenciones

De las 200 tareas, 58 (29 %) fueron resolubles por el detector local sin una llamada al LLM, alcanzando 100 % de precisión dado que los manejadores producen salida determinista. Las 142 tareas dependientes del LLM mostraron 85 % de precisión, confirmando que las tareas más difíciles se delegan apropiadamente a GPT-4o.

4.6 Estudio Piloto con Usuarios

Un estudio piloto con tres usuarios novatos en GNU/Linux—muestra que reconocemos insuficiente para generalización estadística—aportó hallazgos cualitativos que orientaron iteraciones de diseño. Los participantes valoraron la ejecución vía lenguaje natural, las explicaciones educativas (calificadas con 4.4/5.0) y la retroalimentación visual de seguridad. La retroalimentación constructiva identificó confusión con la sintaxis verbal (lo cual motivó la implementación de la acción `clarify`), frustración ante confirmaciones de seguridad excesivas y dificultad con mensajes de error. La transcripción por Whisper promedió 1–3 s; la detección de palabra de activación por Vosk respondió en menos de 200 ms; el ciclo completo voz-a-respuesta promedió 4–8 s (LLM) y 1–2 s (local).

5. DISCUSIÓN

5.1 Implicaciones de Accesibilidad y Costo

La aceleración de $9.4\times$ beneficia contextos educativos—donde estudiantes pueden concentrarse en conceptos en lugar de sintaxis [Norman, 2013]—y profesionales no técnicos. El soporte multidistribución implica que un usuario migrando de Ubuntu a Fedora no necesita reaprender comandos. Desde la perspectiva latinoamericana, una herramienta gratuita que acepta español reduce simultáneamente barreras lingüísticas, económicas y técnicas [CEPAL, 2023, Warschauer, 2004]. A precios actuales de GPT-4o, 50 solicitudes diarias cuestan $\sim \$4.50$ /mes; el detector local reduce esto en $\sim 30\%$ eliminando 500–1 500 ms de latencia. Investigamos la viabilidad de un modelo local (DeepSeek-R1-0528-Distill-8b) [DeepSeek AI, 2025] para eliminar completamente la dependencia de la nube.

5.2 Limitaciones

1. **Estudio piloto preliminar:** El piloto con tres participantes es insuficiente para validación estadística; un estudio formal con muestra ampliada siguiendo ISO 9241-210 [International Organization for Standardization, 2019] es necesario.
2. **Solicitudes compuestas y altamente ambiguas:** Cuando falta un parámetro requerido—como un directorio destino, nombre de archivo o desambiguación entre archivos con nombre idéntico—el sistema dispara una acción `clarify` que presenta una pregunta de seguimiento al usuario antes de proceder. Sin embargo, este mecanismo de turno único no cubre todos los escenarios de ambigüedad: formulaciones muy vagas (ej., “borra los grandes”) aún pueden evadir la clarificación y producir un comando

incorrecto o fallo silencioso. Además, solicitudes compuestas que encadenan múltiples operaciones en una sola expresión (ej., “busca archivos grandes, muévelos al respaldo y comprime la carpeta”) no se descomponen en pasos atómicos verificables, ya que la arquitectura procesa una acción por entrada de usuario. Ambas limitaciones fueron observadas durante las pruebas piloto. Extender la clarificación a diálogos multiturnos e implementar descomposición secuencial de comandos son prioridades del trabajo futuro.

3. **Pruebas en una sola distribución:** Las pruebas sistemáticas se condujeron únicamente en Ubuntu 22.04 pese a soportar seis familias.
4. **Sesgo del evaluador:** Las 200 tareas fueron compiladas por el desarrollador, con potencial sesgo de confirmación inherente.
5. **Sin líneas base comparativas:** No se incluyó una comparación cuantitativa contra otros marcos de comandos por voz.

5.3 Trabajo Futuro

Las mejoras planificadas incluyen: (a) expansión de la detección local de intenciones mediante análisis de registros de interacción; (b) extensión del mecanismo existente de clarificación de turno único (`clarify`) hacia diálogos multiturnos y descomposición secuencial de comandos para solicitudes compuestas; (c) un estudio formal con una muestra más amplia de participantes; (d) investigación de LLMs locales para operación completamente fuera de línea [DeepSeek AI, 2025]; (e) comparación cuantitativa contra líneas base (ShellGPT, NL2Bash); (f) completar y robustecer el componente de explicaciones educativas detalladas (desgloses bandera por bandera) parcialmente implementado; y (g) liberación como software libre.

6. CONCLUSIONES

Hemos presentado BOTAS, una interfaz conversacional para la gestión de sistemas GNU/Linux que encarna los principios de la Inteligencia Artificial Centrada en el Humano. Al traducir lenguaje natural a comandos de terminal validados, BOTAS alcanza 89 % de precisión en generación de comandos, 100 % de detección de seguridad y una aceleración de 9.4× en completación de tareas respecto a métodos manuales.

Las contribuciones clave más allá de la versión sometida a revisión ciega incluyen: un módulo de detección local de intenciones que reduce la dependencia del LLM en aproximadamente 30 %; detección automática de distribución que habilita portabilidad multiplataforma en seis familias de GNU/Linux; un indicador visual de estado que mejora la experiencia de interacción por voz; registro estructurado de solicitudes para auditoría; e infraestructura de empaquetado para instalación sin fricciones vía paquetes `.deb` y `.rpm`.

BOTAS demuestra que los modelos de lenguaje de gran escala pueden democratizar el acceso a herramientas computacionales potentes cuando se diseñan con mecanismos de seguridad apropiados, componentes educativos y atención a comunidades desatendidas. El objetivo no es hacer la tecnología más simple, sino más accesible—y en ese proceso, más humana. Planeamos liberar el sistema como software libre y conducir un estudio formal con usuarios para validar su efectividad educativa.

REFERENCIAS

- Alpha Cephei. Vosk speech recognition toolkit. <https://alphacephei.com/vosk/>, 2023.
- S. Amershi, D. Weld, M. Vorvoreanu, A. Fourney, B. Nushi, P. Collisson, J. Suh, S. Iqbal, P. N. Bennett, K. Inkpen, J. Teevan, R. Kikin-Gil, and E. Horvitz. Guidelines for human–AI interaction. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*, pages 1–13, 2019. doi: 10.1145/3290605.3300233.
- D. Amodei, C. Olah, J. Steinhardt, P. Christiano, J. Schulman, and D. Mané. Concrete problems in AI safety. *arXiv preprint arXiv:1606.06565*, 2016.
- T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, 2020.

- CEPAL. Panorama digital de América Latina y el Caribe 2023: Avances y desafíos de la transformación digital. Technical report, Comisión Económica para América Latina y el Caribe, Naciones Unidas, 2023.
- H. Chase. LangChain: Building applications with LLMs. GitHub Repository, <https://github.com/langchain-ai/langchain>, 2022.
- M. Chen, J. Tworek, H. Jun, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- DeepSeek AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. Technical report, DeepSeek AI, 2025. arXiv:2501.12948.
- A. Dix, J. Finlay, G. D. Abowd, and R. Beale. *Human–Computer Interaction*. Pearson Education, 3rd edition, 2004.
- International Organization for Standardization. ISO 9241-210:2019 — ergonomics of human–system interaction — part 210: Human-centred design for interactive systems. Technical report, ISO, 2019.
- S. Jenson. The future of desktop UX: Breaking the cycle of stagnation. Invited keynote, Interaction Design Forum, 2024.
- X. V. Lin, C. Wang, L. Zettlemoyer, and M. D. Ernst. NL2Bash: A corpus and semantic parser for natural language interface to the Linux operating system. In *Proceedings of the 11th International Conference on Language Resources and Evaluation (LREC 2018)*, 2018.
- E. Luger and A. Sellen. “like having a really bad PA”: The gulf between user expectation and experience of conversational agents. In *Proceedings of the 2016 CHI Conference on Human Factors in Computing Systems*, pages 5286–5297, 2016. doi: 10.1145/2858036.2858288.
- B. A. Myers, A. J. Ko, and M. M. Burnett. Invited research overview: End-user programming. In *Extended Abstracts of the 2004 CHI Conference on Human Factors in Computing Systems*, pages 75–76, 2004.
- J. Nielsen. *Usability Engineering*. Morgan Kaufmann, 1994.
- D. A. Norman. *The Design of Everyday Things*. Basic Books, 1988.
- D. A. Norman. *The Design of Everyday Things: Revised and Expanded Edition*. Basic Books, 2013.
- OpenAI. GPT-4 technical report. Technical report, OpenAI, 2023. arXiv:2303.08774.
- OpenAI. GPT-4o system card. <https://openai.com/index/gpt-4o-system-card/>, 2024.
- A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever. Robust speech recognition via large-scale weak supervision. In *Proceedings of the 40th International Conference on Machine Learning*, 2023. Preprint 2022.
- E. S. Raymond. *The Art of Unix Programming*. Addison-Wesley, 2003.
- T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36, 2024.
- B. Shneiderman. Human-centered artificial intelligence: Reliable, safe & trustworthy. *International Journal of Human–Computer Interaction*, 36(6):495–504, 2020. doi: 10.1080/10447318.2020.1741118.
- B. Shneiderman. *Human-Centered AI*. Oxford University Press, 2022.
- B. Shneiderman and C. Plaisant. *Designing the User Interface: Strategies for Effective Human–Computer Interaction*. Addison-Wesley, 5th edition, 2010.
- The GTK Team. GTK — the GIMP toolkit. <https://www.gtk.org/>, 2023.
- Top500.org. Operating system family / Linux. <https://www.top500.org/statistics/details/osfam/1/>, 2023.
- W3Techs. Usage statistics of Linux for websites. <https://w3techs.com/technologies/details/os-linux>, 2023.
- L. Wall, T. Christiansen, and J. Orwant. *Programming Perl*. O’Reilly Media, 3rd edition, 2000.
- M. Warschauer. *Technology and Social Inclusion: Rethinking the Digital Divide*. MIT Press, 2004.
- J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35, 2022.
- J. O. Wobbrock, S. K. Kane, K. Z. Gajos, S. Harada, and J. Froehlich. Ability-based design: Concept, principles and examples. In *ACM Transactions on Accessible Computing*, volume 3, pages 1–27, 2011. doi: 10.1145/1952383.1952384.
- S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.